

**INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA**
SUL-RIO-GRANDENSE
Campus Passo Fundo

Bacharelado em Ciência da Computação
Arquitetura de Computadores I

Conteúdo 3:

Comandando o Hardware:

operações, operandos, instruções, representações, endereçamento

Prof. Petry, Carlos A



Baseado no livro:
Organização e Projeto de Computadores
David Patterson

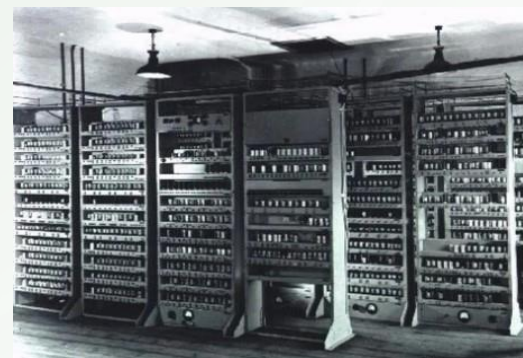
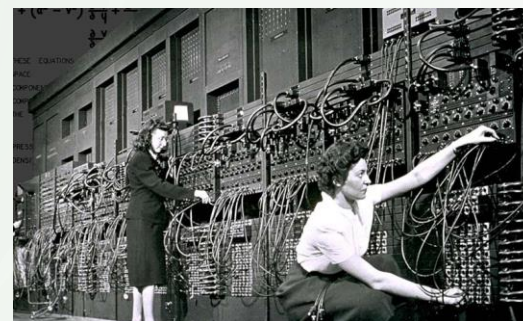
INTRODUÇÃO: INSTRUÇÕES E CARACTERÍSTICAS

- Para controlar o hardware: **precisamos usar a linguagem digital binária**
- Processadores são **comandados através de** “palavras” chamadas de **instruções**
- Existem diversas **instruções**: **aritméticas**, **lógicas**, **E/S**, etc
- Agrupando **todas as instruções** temos o **Conjunto de Instruções** (Instruction Set)
- Conjunto de Instruções: **vocabulário de comandos** entendidos por uma determinada **arquitetura** (processador ou família)
- Similaridade do conjunto de instruções entre diferentes computadores*:
 - Devido aos computadores serem construídos baseando-se em **princípios tecnológicos semelhantes**
→ circuitos digitais
 - Devido à diversas **operações básicas** presentes na maioria dos computadores: aritméticas, logico-comparativas, desvios, E/S
 - Ao projeto de hardware e compilador **maximizar desempenho e minimizar custo**

*O termo computadores neste contexto se refere a processadores

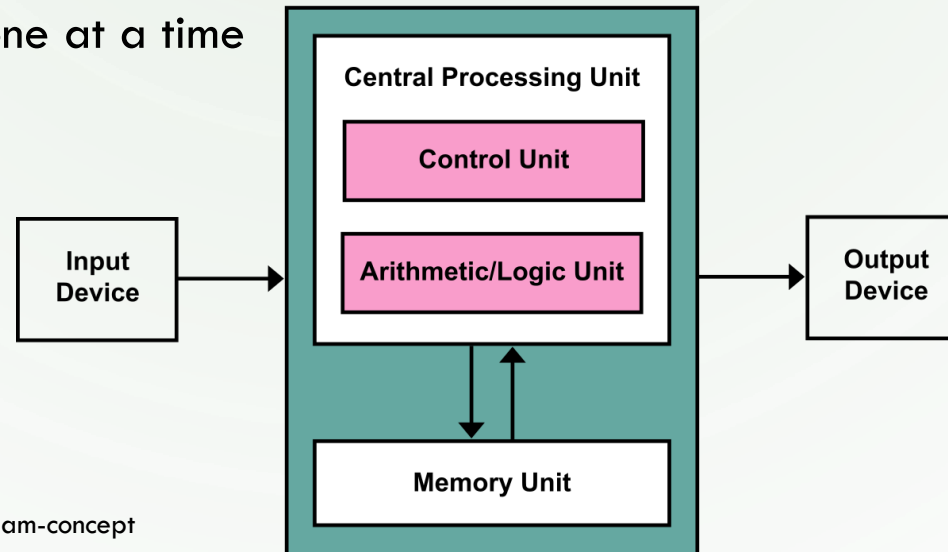
INTRODUÇÃO: ARMAZENAMENTO

- Os primeiros computadores não possuíam armazenamento
- Computadores como o ENIAC (8/1944)
 - Eram programados diretamente em linguagem de máquina
 - A programação era feita através da conexão de cabos na unidade de controle
 - Operações a executar eram definidas por conexões físicas
 - Usava o sistema decimal como numeração
- Já computadores como EDVAC e UNIVAC (4/1946, 3/1951) passaram usar o conceito de programa residente me memória
 - Conceito desenvolvido por John von Neumann
 - Usava o sistema binário como numeração
 - Realizava o processamento em série
 - Operações a executar localizavam-se em memória
 - Operações definitas por um conjunto de instruções



INTRODUÇÃO: ARMAZENAMENTO

- Conceito de **Programa Armazenado: von Neumann Architecture**
 - Instruções e dados de diversos tipos podem ser armazenados na memória como números
 - ¹A computer that stores instructions in its memory to enable it to perform a variety of tasks in sequence or intermittently
 - ²A computer that stores program instructions in electronically or optically accessible memory
 - ³A program must be in main memory in order for it to be executed. The instructions are fetched, decoded and executed one at a time



¹ <https://www.britannica.com/technology/stored-program-concept>

² https://en.wikipedia.org/wiki/Stored-program_computer

³ https://en.wikibooks.org/wiki/A-level_Computing/AQA/Paper_2/Fundamentals_of_computer_organisation_and_architecture/The_stored_program_concept

⁴ https://en.wikipedia.org/wiki/Von_Neumann_architecture

INTRODUÇÃO - MIPS:

CONJUNTO DE INSTRUÇÕES EXEMPLO

- Assembly usada como estudo de caso
- Conjunto de instruções MIPS

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1,\$s2,\$s3	$s1 = s2 + s3$	Three register operands
	subtract	sub \$s1,\$s2,\$s3	$s1 = s2 - s3$	Three register operands
	add immediate	addi \$s1,\$s2,20	$s1 = s2 + 20$	Used to add constants
Data transfer	load word	lw \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Word from memory to register
	store word	sw \$s1,20(\$s2)	$\text{Memory}[s2 + 20] = s1$	Word from register to memory
	load half	lh \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Halfword memory to register
	load half unsigned	lhu \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Halfword memory to register
	store half	sh \$s1,20(\$s2)	$\text{Memory}[s2 + 20] = s1$	Halfword register to memory
	load byte	lb \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Byte from memory to register
	load byte unsigned	lbu \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Byte from memory to register
	store byte	sb \$s1,20(\$s2)	$\text{Memory}[s2 + 20] = s1$	Byte from register to memory
	load linked word	ll \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Load word as 1st half of atomic swap
	store condition. word	sc \$s1,20(\$s2)	$\text{Memory}[s2 + 20] = s1; s1 = 0 \text{ or } 1$	Store word as 2nd half of atomic swap
	load upper immed.	lui \$s1,20	$s1 = 20 * 2^{16}$	Loads constant in upper 16 bits
Logical	and	and \$s1,\$s2,\$s3	$s1 = s2 \& s3$	Three reg. operands; bit-by-bit AND
	or	or \$s1,\$s2,\$s3	$s1 = s2 s3$	Three reg. operands; bit-by-bit OR
	nor	nor \$s1,\$s2,\$s3	$s1 = \sim (s2 s3)$	Three reg. operands; bit-by-bit NOR
	and immediate	andi \$s1,\$s2,20	$s1 = s2 \& 20$	Bit-by-bit AND reg with constant
	or immediate	ori \$s1,\$s2,20	$s1 = s2 20$	Bit-by-bit OR reg with constant
	shift left logical	sll \$s1,\$s2,10	$s1 = s2 \ll 10$	Shift left by constant
	shift right logical	srl \$s1,\$s2,10	$s1 = s2 \gg 10$	Shift right by constant
Conditional branch	branch on equal	beq \$s1,\$s2,25	if ($s1 == s2$) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne \$s1,\$s2,25	if ($s1 \neq s2$) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt \$s1,\$s2,\$s3	if ($s2 < s3$) $s1 = 1$; else $s1 = 0$	Compare less than; for beq, bne
	set on less than unsigned	sltu \$s1,\$s2,\$s3	if ($s2 < s3$) $s1 = 1$; else $s1 = 0$	Compare less than unsigned
	set less than immediate	slti \$s1,\$s2,20	if ($s2 < 20$) $s1 = 1$; else $s1 = 0$	Compare less than constant
	set less than immediate unsigned	sltiu \$s1,\$s2,20	if ($s2 < 20$) $s1 = 1$; else $s1 = 0$	Compare less than constant unsigned
Unconditional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr \$ra	go to \$ra	For switch, procedure return
	jump and link	jal 2500	$ra = PC + 4$; go to 10000	For procedure call

INTRODUÇÃO - MIPS:

CONJUNTO DE INSTRUÇÕES EXEMPLO

- Assembly usada como estudo de caso
- Operando e endereços de memória

Name	Example	Comments
32 registers	<code>\$s0-\$s7, \$t0-\$t9, \$zero, \$a0-\$a3, \$v0-\$v1, \$gp, \$fp, \$sp, \$ra, \$at</code>	Fast locations for data. In MIPS, data must be in registers to perform arithmetic, register <code>\$zero</code> always equals 0, and register <code>\$at</code> is reserved by the assembler to handle large constants.
2^{32} memory words	<code>Memory[0], Memory[4], . . . , Memory[4294967292]</code>	Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers.

OPERAÇÕES DO HARDWARE

- Podemos considerar operações **aritméticas** como as **operações básicas** em computadores
- Exemplo, operação soma em MIPS:
add a, b, c
 - Instrui ao hardware somar os operandos b e c e colocar o resultado em a
- MIPS (e processadores RISC) realizam operações considerando:
 - Uma única operação por ciclo
 - Uso padrão de 3 operandos
 - Operações simples, fundamentais

OPERAÇÕES DO HARDWARE

- Exemplo, para somar os operandos b, c, d, e e armazenar o resultado em a , devemos realizar 3 operações, ou seja instruções:

`add a, b, c # a = b+c`

`add a, a, d # a = a+d (b+c+d)`

`add a, a, e # a = a+e (b+c+d+e)`

- Princípio de projeto 1: **simplicidade favorece a regularidade**
- O exemplo mostra o princípio da simplicidade do hardware:
 - Manter fixo o n° de operandos
 - Manter a operação especializadas

OPERAÇÕES DO HARDWARE

- Exemplo, expressões em linguagem C para MIPS:

`a = b + c;`

`d = a - e;`

- Instruções MIPS equivalente às expressões C:

`add a, b, c # a = b+c`

`sub d, a, e # d = a-e`

- Exemplo, expressões em linguagem C para MIPS:

`f = (g+h) - (i+j)`

- Instruções MIPS equivalente às expressões C desmembrada em operações “binárias”:

`add $t0, g, h # t0 = g+h`

`add $t1, i, j # t1 = i+j`

`sub f, $t0, $t1 # f = t0-t1`

- Exercício:

Apresente 3 exemplos de expressões em C e seu equivalente em MIPS

OPERANDOS EM HARDWARE

- **Operandos** em linguagens de baixo nível são **escassos**, diferente de variáveis em linguagens de alto nível
- **Operando** em hardware são armazenados **em registradores**, dentro do processador, denominados GPR (General Purpose Registers)
- Diferença entre variáveis e registradores
 - Variáveis: rótulos para endereços de memória em linguagem de alto nível
 - Registradores: elementos de memória interna do processador, geralmente Flip-flops, manipulados por linguagens de baixo nível (assembly)

OPERANDOS EM HARDWARE

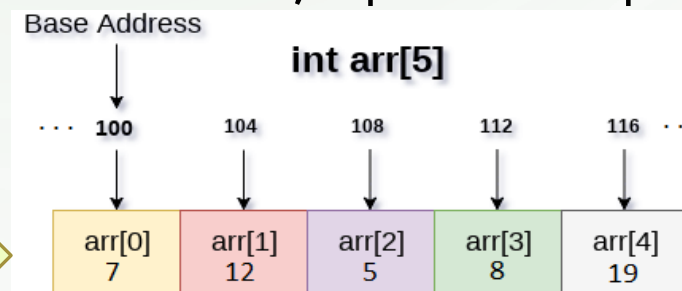
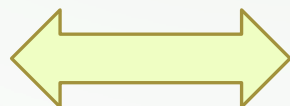
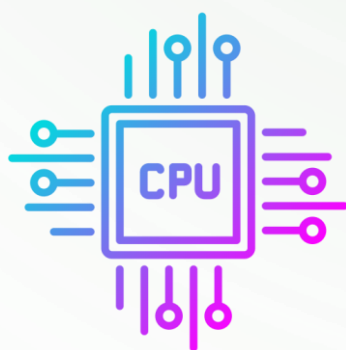
- **Palavra** (word)
 - Unidade de acesso de um computador
 - Corresponde ao número de vias (fios) do barramento (Bus)
 - No MIPS o barramento possui 32 bits
- Em MIPS
 - Todos os **registradores** possuem **32 bits**
 - Existem 32 registradores (memória interna do processador), denominados **Banco de Registradores** (BR)
- Princípio de projeto 2: **menor significa mais rápido**
 - Mantendo elementos simples no processador proporciona um hardware mais rápido
 - Exemplo, uso de apenas 32 registradores no MIPS
 - Existe um compromisso entre: **mais recursos e mais simplicidade**
 - “Menor” pode ser obtido por **processo de fabricação** de menor dimensão

OPERANDOS EM MEMÓRIA

- Linguagens de alto nível
 - Usam **muitas** variáveis
 - Possuem **estruturas** de dados simples e **complexas** (vetores, matrizes, registros)
- Linguagens de baixo nível (assembly)
 - Possuem um número **limitado** de registradores (32)
 - Armazenam apenas uma palavra (word, 32b) por vez
- Questão:
 - Como **compatibilizar** o **uso de dados** entre linguagens de alto e baixo nível?
- Resposta
 - Os **dados** podem ficar armazenados na **memória principal** , maior que registradores
 - Da **memória** são transferidos para **registradores** e de volta para **memória**

OPERANDOS EM MEMÓRIA PRINCIPAL

- Em arquiteturas modernas (RISC) operações ocorrem apenas com registradores (não com a memória)
- Operações **aritméticas** (e outras) buscam e colocam dados de/em **registradores**
- Operações de entrada e saída (I/O) trazem e levam dados da/para **memória**
- Operações I/O buscam e colocam dados em memória com base em **endereços**
- Endereço de memória
 - **Valor numérico** indicando uma posição de **memória**, representada por um grande **vetor**



OPERANDOS EM MEMÓRIA PRINCIPAL

- Instruções de transferência de dados
 - Comando que **move** dados entre **memória e registradores** (I/O)
 - **Load**: instrução para transfere dados da memória **para registrador**
 - **Store**: instrução para transferência de dados do registrador **para memória**
- Endereço de memória
 - **Valor** que indica o **local** do elemento de **dado** na sequência **de células de memória**
- Conteúdo de memória
 - **Valor** contido **dentro** de um elemento de dado na **célula de memória**

OPERANDOS EM MEMÓRIA PRINCIPAL

- Em geral, programas acessam posições em memória para manipular dados
 - Para **acessar memória** precisamos sempre **indicar o endereço** alvo
 - Para realizar operações precisamos do dados no registrador
 - Portanto, para realizar operações envolvendo memória

- Exemplo, expressões de atribuição em C para MIPS :

`g = h + A[2];`

- Instruções **MIPS equivalente** à expressão C:

`lw $t0, 8($s3)      # t0 = Mem[s3+8], s3->A[0]`

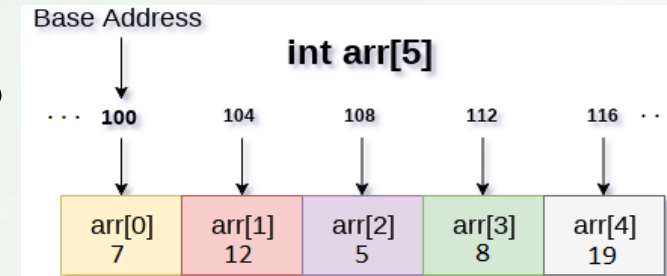
`add $s1, $s2, $t0   # s1<-soma, s2=h (em registrador)`

- Exercício:

Apresente 3 exemplos de expressões com acesso à memória em C e seu equivalente em MIPS

OPERANDOS EM MEMÓRIA PRINCIPAL

- ✓ Estrutura de dados em memórias
- Estrutura de dados, como vetores, facilmente superam o número de registradores
- Para manipular estruturas o compilador define o endereço da estrutura como o local de início da estrutura (endereço base), por exemplo o vetor de inteiros `arr` possui endereço inicial 100:
- Endereços de estruturas são alinhados conforme o tamanho do dado
 - Inteiros ocupam, em geral, 32 bits ou 4 bytes
- Assim, `arr[0]` está no endereço 100, `arr[1]` está no 104, ...



OPERANDOS EM MEMÓRIA PRINCIPAL

✓ Restrição de alinhamento em memória

- Em MIPS, **endereços** precisam ser **múltiplos de 4** (32b), precisam estar **alinhados**
- A manipulação de dados podem ocorrer a nível de byte (8b), half-word (16b) e word (32b)
- Porém, acessos à memória devem ocorrer como múltiplos de 4

• ¹Armazenamento em memória podem seguir dois modelos:

1. **Little endian:**

armazena o byte **menos** significativo da palavra (word) no **menor** endereço de memória

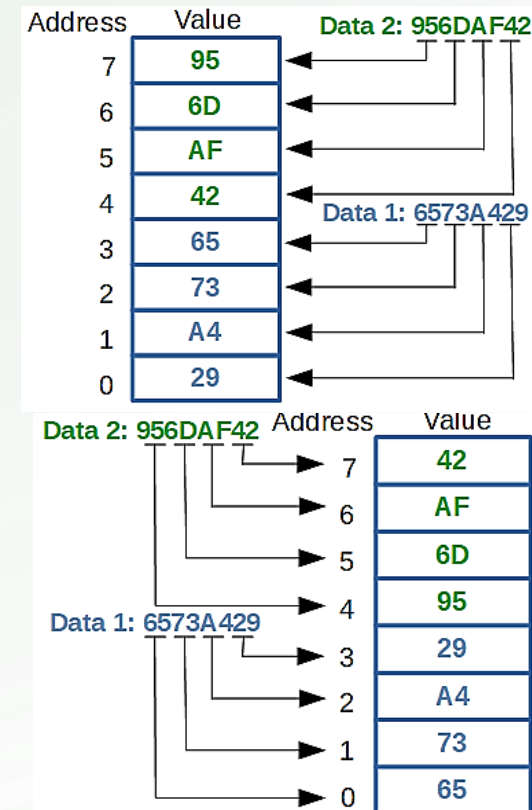
2. **Big endian:**

armazena o byte **mais** significativo da palavra (word) no **menor** endereço de memória

• **MIPS** usa o modelo **big** endian

• Exercícios, mostre 3 exemplos de vetores: big e little endian

¹ <https://en.wikipedia.org/wiki/Endianness>



OPERANDOS EM MEMÓRIA PRINCIPAL

- ✓ Restrição de alinhamento em memória
- Acessos à **endereços** de **memória** são expressos em **binário**
- Portanto, **RAM** (MP) são sempre baseados na **base 2** (binária)
 - 1 mega byte (MiB, mebibyte) equivale à **1048576**, não à 1.000.000
 - 1 giga byte (GiB, gibibyte) equivale à **1073741824**, não à 1.000.000.000
 - Assim, a **capacidade** da memória **RAM** é sempre um fator de **base 2**:
4 MiB equivalem a 4.194.304 bytes e 4GiB equivale a 4.294.967.296

OPERANDOS EM MEMÓRIA PRINCIPAL

✓ Operações em memória

- Dispositivos de armazenamento **armazenam Dados**
- Dispositivos de armazenamento **representam a memória** computacional
- Existem 2 tipos (básicos) de **operações sobre a memória**
 - **Leitura e Escrita**

OPERANDOS EM MEMÓRIA PRINCIPAL

✓ Operações em memória

- Operações em memória são consideradas do ponto de vista do processador
 - **Processador lê** dados da memória e **escreve** dados na memória
- Como é o processador quem realiza operações, consideramos:
 - **Leitura** como uma carga de dados → **Load**
 - **Escrita** como um armazenamento de dados → **Store**
- Operações de **leitura** representam a **saída** de dados na **memória** principal, enquanto operações de **escrita** representam **entrada** de dados na **memória**

OPERANDOS EM MEMÓRIA PRINCIPAL

✓ Instruções de transferência de dados (MP)

- Operações de leitura em memória

- Realiza a carga (**load**) de dados para o **processador**
- Assim, LOAD é a instrução que carrega e dados
- Em **MIPS** a instrução para leitura em MP é **lw** (ou **lh** ou **lb**)

- Operações de escrita em memória

- Realiza o armazenamento (**store**) de dados a partir do **processador**
- Assim, STORE é a instrução que armazena dados
- Em **MIPS** a instrução par escrita em MP é **sw** (ou **sh** ou **sb**)

- **Load** e **Store** podem transferir uma palavra (**word**), meia-palavra (**half**) ou 8 bits (**byte**), mas sempre **manipula 32 bits**: o tamanho da **Word** da arquitetura MIPS

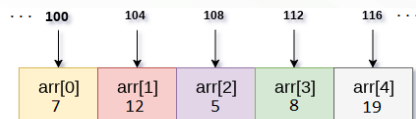
OPERANDOS EM MEMÓRIA PRINCIPAL

- ✓ Exemplo 1: programa com transferência de dados (MP)
- Considere o comando que soma uma variável e um elemento de um vetor e atribui o resultado a outro elemento do vetor **int**: $A[12] = h + A[8]$
 - O endereço base do vetor A está armazenado em $\$s3$ e o endereço da variável* h está armazenado em $\$s2$ (lembrete: valores são inteiros de 32 bits)
 - As instruções MIPS que executam este comando poderia ser as seguintes:

```
lw $t0, 32($s3) # $t0 recebe o valor do vetor em A[8] (A<-$s3)
add $t0, $s2, $t0 # t0 recebe h + A[8] (h<-$s2)
sw $t0, 48($s3) # armazena a soma ($t0) no vetor em A[12]
```

Versão completa e correta:

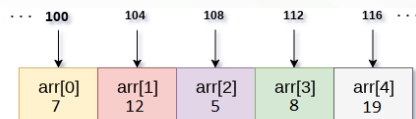
```
lw $t1, 0($s2) # $t1 recebe o valor da variável h (h<-$s2)
lw $t0, 32($s3) # $t0 recebe o valor do vetor em A[8] (A<-$s3)
add $t2, $t1, $t0 # t0 recebe h + A[8] (h<-$s2)
sw $t2, 48($s3) # armazena a soma ($t0) no vetor em A[12]
```



OPERANDOS EM MEMÓRIA PRINCIPAL

- ✓ Exemplo2: programa com transferência de dados (MP)
- Considere o comando onde uma variável receba o endereço que aponte para o 4º elemento (índice 3) de um vetor : $pA = A + 12$
 - O endereço base do vetor A está armazenado em $\$s3$, a variável pA será armazenada em $\$s2$ e a constante 12 fará parte da instrução (lembrete: valores são inteiros de 32 bits)
 - As instruções MIPS que executam este comando são as seguintes:

```
lw $t0, 0($s3) # $t0 recebe o valor do vetor em A[0] (A<-$s3)  
addi $t0, $s3, 12 # $t0 recebe a soma do endereço base de A com 12  
sw $t0, 0($s2) # armazena o endereço de A[12] em pA (pA<-$s2)
```



OPERANDOS EM MEMÓRIA PRINCIPAL

✓ Instruções de transferência de dados (MP)

- Como vimos, existem 32 registradores 0-31, denominados da seguinte forma:

Registrador	Nome	Observações
\$0	\$zero ou \$r0	Sempre zero (hardware wired to zero)
\$1	\$at	Reservado para assembler
\$2, \$3	\$v0, \$v1	1º e 2º valores de retorno de função
\$4, ..., \$7	\$a0, ..., \$a3	1º e 2º argumentos de função
\$8, ..., \$15	\$t0, ..., \$t7	Registradores Temporários (Temp)
\$16, ..., \$23	\$s0, ..., \$s7	Registradores Preservados (Saved)
\$24, \$25	\$t8, \$t9	Registradores Temporários, adicionais (Temp)
\$26, \$27	\$k0, \$k1	Reservado para o kernel do SO
\$28	\$gp	Global pointer
\$29	\$sp	Stack pointer
\$30	\$fp	Frame pointer
\$31	\$ra	Return address

OPERANDOS EM MEMÓRIA PRINCIPAL

- ✓ Instruções de transferência de dados (MP)
- Registradores no MIPS usam o método de **endereçamento**:
registrado-base + deslocamento (**offset base-register**)
 - Adequado para acesso à arrays (vetores/matrizes)
 - Utiliza o **endereço base** do array e adiciona um **índice** referente ao tipo de dado
 - Por exemplo, na instrução `lw $t0, 32($s3)`, `$s3` contém o endereço base do array, enquanto `32` representa o deslocamento (offset) para o 9º elemento (índice 8): $32/4$, lembre valores `inteiros` usam 32 bits
- Exercícios, mostre 3 exemplos de expressões matemáticas com acesso à memória e as instruções MIPS equivalentes

NÚMEROS POSITIVOS E NEGATIVOS

- ✓ Representação de números com e sem sinal
- Seres **humanos** manipulam números em base **decimal** (10)
- Sistemas **computacionais** manipulam números em base **binária** (2)
 - Números **binários** são representados por dois **símbolos**: 0 e 1 (bits)
 - **Tecnologicamente** representamos binários através de dois níveis: baixo e alto
 - O mecanismo tecnológico mais comum para **representar binários** é usando **eletricidade**
- Em **MIPS** os valores são armazenados segundo o método **Big endian**
 - Em uma palavra (**word**) MIPS temos o **LSD** mais à **direta** e o **MSD** mais à **esquerda**, observando o número, coincidindo com a notação decimal
 - Na memória, o **LSD** está localizando após do **MSD** (endereço crescente)
- Exemplo: faixa de numerações para 32 bits

0 -> 4294967295 **ou** -2147483648 -> 0 -> 2147483647

NÚMEROS POSITIVOS E NEGATIVOS

✓ Representação de números com e sem sinal

- Usando 32 bits podemos representar 2^{32} diferentes valores

- Exemplo: valores positivos binário/decimal para 32 bits (0 a $2^{32}-1$)

0000 0000 0000 0000 0000 0000 0000 0000₂ = 0₁₀

0000 0000 0000 0000 0000 0000 0000 0001₂ = 1₁₀

0000 0000 0000 0000 0000 0000 0000 0010₂ = 2₁₀

. . .

1111 1111 1111 1111 1111 1111 1111 1101₂ = 4.294.967.293₁₀

1111 1111 1111 1111 1111 1111 1111 1110₂ = 4.294.967.294₁₀

1111 1111 1111 1111 1111 1111 1111 1111₂ = 4.294.967.295₁₀

- Observe as primeiras números acima, todos possuem 0s à esquerda. Por quê?
- A resposta está relacionada à arquitetura: uso de palavras (words)

NÚMEROS POSITIVOS E NEGATIVOS

- ✓ Representação de números com e sem sinal
- Além de **números** positivos, usamos também **negativos**
- Precisamos de uma **notação** (padrão) para **representar** e distinguir valores positivos e negativos
- Em aritmética tradicional usamos a seguinte notação:
 - Sinal (+-) colocado à extrema esquerda do dígito mais significativo
 - Por exemplo: +123 e -384

NÚMEROS POSITIVOS E NEGATIVOS

✓ Possíveis notações:

- Acrescentar um sinal indicativo de positivo ou negativo
 - Podemos usar **um bit** para indicar o **sinal**, chamamos de **sinal magnitude**
 - Problemas com esta representação:
 - **Onde** (em que bit) colocar sinal magnitude?
(**a**) à esquerda do dígito mais significativo, (**b**) no MSD, (**c**) no LSD, (**d**) no cento da palavra, (**e**) em um bit adicional ?
 - **Operações aritméticas**, como soma e multiplicação (feitas **em HW**), precisam de um padrão rígido para realizar as operações
 - O sinal magnitude em um **bit adicional** produz **zero** (0) **positivo** e **negativo**
- Assim, sinal magnitude não representa uma solução viável

NÚMEROS POSITIVOS E NEGATIVOS

✓ Possíveis notações:

- Representação matemática para positivo e negativo
 - Qual o resultado da subtração de número maior (grande) de um número menor (pequeno)?
 - Invertemos os termos, realizamos a subtração e colocamos negativo na resposta
 - Caso o dígito do subtraendo for maior que o minuendo, tomamos emprestado dos dígitos à esquerda
 - Exemplo: $2 - 8 = 8 - 2$ com resultado negativo $\rightarrow 6 \rightarrow -6$
 - Este mecanismo não funciona para valores binários sinal magnitude

NÚMEROS POSITIVOS E NEGATIVOS

✓ Possíveis notações:

- Notação em complemento de 2

- **Divide** a faixa de valores em **duas parte**: 1ª metade **positivos**, 2ª metade **negativos**
- Possibilita apenas uma instância do **valor zero** (0)
- Possui um valor **negativo a mais** que o positivo
- Todos os valores negativos possuem o **MSD 1**, simplificando o HW, chamado **bit de sinal**
- O número decimal convertido para **binário de 32 bits** equivale ao seguinte **polinômio**:

$$(x_{31} * -2^{31}) + (x_{30} * 2^{30}) + (x_{29} * 2^{29}) + \dots + (x_2 * 2^2) + (x_1 * 2^1) + (x_0 * 2^0)$$

- **Por exemplo:**

$$1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1100_2 = 4_{10}$$

$$\rightarrow (1 * -2^{31}) + (1 * 2^{30}) + (1 * 2^{29}) + \dots + (1 * 2^2) + (0 * 2^1) + (0 * 2^0)$$

- A conversão de um valor binário positivo X para seu equivalente negativo pode ser feito:

- $\text{Not}(x) + 1 \rightarrow (\text{negação bit a bit}) + 1$ / No exemplo acima resulta: 0000 ... 0100

NÚMEROS POSITIVOS E NEGATIVOS

✓ Possíveis notações:

- Faixa de valores positivos/negativos binário/decimal para palavra de 32 bits, em complemento de 2:

0000 0000 0000 0000 0000 0000 0000 0000₂ = 0₁₀
0000 0000 0000 0000 0000 0000 0000 0001₂ = 1₁₀
.
.
.
0111 1111 1111 1111 1111 1111 1111 1110₂ = 2.147.483.646₁₀
0111 1111 1111 1111 1111 1111 1111 1111₂ = 2.147.483.647₁₀

1000 0000 0000 0000 0000 0000 0000 0000₂ = -2.147.483.648₁₀
1000 0000 0000 0000 0000 0000 0000 0001₂ = -2.147.483.647₁₀
.
.
.
1111 1111 1111 1111 1111 1111 1111 1110₂ = -2₁₀
1111 1111 1111 1111 1111 1111 1111 1111₂ = -1₁₀

- Exercício: (use completo de 2 e palavra 32 bits)
 - Converta 4 números decimais positivos para binário
 - Apresente os 4 números binário em seu equivalente negativo
 - Qual o decimal equivalente para a palavra binária de 64 bits?

1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1000₂

Como ficaria a faixa de valores, decimal↔binário, se usássemos 5 bits?

REPRESENTANDO INSTRUÇÕES

- ✓ Representação de **instruções de computador** (processador):
 - Seres **humanos** escrevem comandos em um formato: **alfa-textual**
 - Porém, **computadores** reconhecem comandos (instruções) diferente: **binário**
 - Instruções são **representadas como números** (binários), em geral **compostas** por **campos**
 - Campos representam:
 - Operação, registradores, endereços/deslocamentos, constantes, etc
 - Formato de instrução
 - **Representação de uma instrução** composta por combinações de campos
 - Existem vários diferentes **formatos de instruções**

REPRESENTANDO INSTRUÇÕES

✓ Representação de **instruções de computador** (processador):

- Exemplo: a instrução MIPS `add $t0, $s1, $s2` fica assim representada:

Dec	0	17	18	8	0	32
Bin	000000	10001	10010	01000	00000	100000
Bits	6	5	5	5	5	6
Instr.	arit.	\$s1	\$s2	\$t0	-	add

- Linguagem de máquina

- Representação de instruções em formato numérico
- Instruções no computador são armazenadas em binário
- Para tornar mais simples, costuma-se usar o formato numérico hexadecimal
- Para o exemplo acima a instrução de soma ficaria representada como 02324020_{16} :

Bin	000000	10001	10010	01000	00000	100000
ou	0000	0010	0011	0010	0100	0000 0010
Hexa	02	32	40	20		

REPRESENTANDO INSTRUÇÕES

- ✓ Representação de **instruções de computador** (processador):
- **Campos** na representação de **instruções MIPS**:

Campo	op	rs	rt	rd	shamt	funct
Bits	6	5	5	5	5	6

- op : operação básica (principal) da instrução → **operation code**
- rs : 1º registrador **origem** (fonte)
- rt : 2º registrador **origem** (fonte)
- rd : registrador **destino** (resultado)
- shamt: quantidade de deslocamento*
- funct: **função** secundária da instrução → **function code**

REPRESENTANDO INSTRUÇÕES

- ✓ Representação de **instruções de computador** (processador):
- Porém, existem casos que precisamos **campos mais longos** (+bits)

Campo	op	rs	rt	Immediate/offset	
Bits	6	5	5	16	

- **Campos** na representação de **instruções MIPS**:
- Exemplo, na instrução `load`:
 - `lw $t0, 24($s3)` → precisamos 2 **registradores** e uma **constante** (16b)
 - Usando **5 bits** para a constante ficamos limitados ao valor **máximo 32** (2^5)
 - Permitindo leituras para apenas **32 posições** a partir do **endereço base** (insuficiente)
 - Poderíamos aumentar a quantidade de bits: $32 \rightarrow 40$

REPRESENTANDO INSTRUÇÕES

- ✓ Representação de **instruções de computador** (processador):
- Princípio de projeto 3: um bom projeto exige bons compromissos
 - Manter fixo o tamanho das instruções
 - Criar diferentes formatos de instruções

- O formato anterior (soma) é então denominado: **formato R** (registrador)

Campo	op	rs	rt	rd	shamt	funct
Bits	6	5	5	5	5	6

- O Formato para instruções I/O e immediatos denomina-se: **formato I** (imediato),
contendo os seguintes campos:

Campo	op	rs	rt	imediato/constante
Bits	6	5	5	16

- Assim, a constante (imediato) passa a ter 16 bits chegando ao **máximo 32768** (2^{16})
- Permitindo **32768 posições** a partir do **endereço base**

REPRESENTANDO INSTRUÇÕES

✓ Representação de **instruções de computador** (processador):

- Exemplo: a instrução MIPS `lw $t0, 24($s3)` fica assim representada:

Dec	35	19	8	24
Bin	100011	10011	01000	0000 0000 0001 1000
Bits	6	5	5	16
Instr.	lw	\$s3	\$t0	imediato

- Linguagem de máquina

- Para o exemplo acima a **instrução de carga** ficaria representada como `8E680018`₁₆:

Bin	100011	10011	01000	0000	0000	0001	1000
ou	1000	1110	0110	1000	0000	0000	0001 1000
Hexa	8E	68	00	18			

REPRESENTANDO INSTRUÇÕES

✓ Expressões e **instruções de computador** (processador):

- Exemplo:

- Programa/expressão:

```
A[360] = h + A[300];           // Endereços base de A em $t1 e o endereço de h em $s2 (inteiros)
```

- Instruções MIPS:

```
lw $t2,0($s2)    # Carrega: $t2 = h
lw $t0,1200($t1) # Carrega: $t0 = A[300]
add $t0,$t2,$t0   # Soma: $t0 = h + A[300]
sw $t0,1440($t1)  # Armazena: A[360] = $t0
```

- Formatos e valores de instruções** do exemplo acima

Instr	op	rs	rt	rd	Shamt offset	funct
lw	35	18	10		0	
lw	35	9	8		1200	
add	0	10	8	8	0	32
sw	43	9	8		1440	

REPRESENTANDO INSTRUÇÕES

✓ Representação de **instruções de computador** (processador):

- Instruções MIPS:

```
lw $t2,0($s2)      # Carrega: $t2 = h
lw $t0,1200($t1)    # Carrega: $t0 = A[300]
add $t0,$t2,$t0     # Soma: $t0 = h + A[300]
sw $t0,1440($t1)    # Armazena: A[360] = $t0
```

- Linguagem de máquina

- Para o exemplo acima a **instrução de carga** ficaria representada como:

lw - Bin	100011	10010	01010	0000000000000000	Hexa
Ou	1000	1110	0100	1010 0000 0000 0000	
Hexa	8E	4A	00	00	8E4A0000 ₁₆
lw - Bin	100011	01001	01000	0000010010110000	
Ou	1000	1101	0010	1000 0000 0100 1011 0000	
Hexa	8D	28	04	B0	8D2804B0 ₁₆
add - Bin	000000	01010	01000	01000 00000 100000	
Ou	0000	0001	0100	1000 0100 0000 0010 0000	
Hexa	01	48	40	20	01484020 ₁₆
sw - Bin	101011	01001	01000	0000010110100000	
ou	1010	1101	0010	1000 0000 0101 1010 0000	
Hexa	AD	28	05	A0	AD2805A0 ₁₆

REPRESENTANDO INSTRUÇÕES

✓ Revisão de **instruções de computador** (processador):

- Resumo e exemplos de instruções MIPS (parte **fixa** e **variável**):

Nome	Formato	Exemplo						Comentários
add	R	0	18	19	17	0	32	add \$s1,\$s2,\$s3
sub	R	0	18	19	17	0	34	sub \$s1,\$s2,\$s3
Tamanho	–	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	Total 32 bits
Formato	R	op	rs	rt	rd	shamt	Function	Aritmético
addi	I	8	18	17		100		addi \$s1,\$s2,100
lw	I	35	18	17		100		lw \$s1,100(\$s2)
sw	I	43	18	17		100		sw \$s1,100(\$s2)
Tamanho	–	6 bits	5 bits	5 bits		16 bits		Total 32 bits
Formato	I	op	rs	rt		imediato		Formato I/O

REPRESENTANDO INSTRUÇÕES: LÓGICAS E DESLOCAMENTO

- ✓ Representação de **instruções de computador** (processador):
 - Em geral, manipulamos **dados múltiplos** de **1,2,4,8,10** Bytes (8,16,32,64,80 bits)
 - Porém, por vezes precisamos **manipular um ou mais bits**
 - Principais **operações** sobre **bits**: **AND, OR, NOT, SHIFT** (deslocamento)
 - **And, Or, Not** são operações lógicas Booleanas
 - **Shift** são operações de deslocamento de bits

REPRESENTANDO INSTRUÇÕES: LÓGICAS E DESLOCAMENTO

✓ Representação de **instruções de computador** (processador):

- Principais operações sobre bits em MIPS:

Operação	Operador C	Operador Java	Instrução MIPS
Shift à esquerda	<<	<<	sll
Shift à direita	>>	>>>	srl
AND bit-a-bit*	&	&	and, andi
OR bit-a-bit*			or, ori
NOT bit-a-bit*	~	~	nor

* **Bitwise**: computando aplicada sobre conjuntos de bit, **opera individualmente sobre cada bit**

Ref: https://en.wikipedia.org/wiki/Bitwise_operation

REPRESENTANDO INSTRUÇÕES: DESLOCAMENTO

- ✓ Representação de **instruções de computador** (processador):
 - Operações de deslocamento: movem bits para a esquerda ou para a direita
 - **Deslocam bits dentro do conjunto** de bits (agrupamento)
 - Exemplo

1. **Deslocamento de 4 bits à esquerda** do conjunto $0000\ 0000\ 0000\ 1001_2 = 9_{10}$

- Resulta em: $0000\ 0000\ 1001\ 0000_2 = 144_{10}$

2. **Deslocamento de 4 bits à direita** do conjunto $0000\ 0000\ 1001\ 0000_2 = 144_{10}$

- Resulta em: $0000\ 0000\ 0000\ 1001_2 = 9_{10}$
- Qual a **relação entre os resultados** dos exemplos 1 e 2: $9 \rightarrow 144$ e $144 \rightarrow 9$?
 - Multiplicação e divisão por potenciações em base 2

REPRESENTANDO INSTRUÇÕES: DESLOCAMENTO

✓ Representação de **instruções de computador** (processador):

- Instruções MIPS de deslocamento:

`sll`: **deslocamento à esquerda** (shift left logical)

`srl`: **deslocamento à direita** (shift right logical)

- Exemplo

- Considere:

- o valor original em `$s0` e o resultado em `$t2` para deslocamento à esquerda, e

- o valor original em `$t2` e o resultado em `$t4` para deslocamento à direita

- Instruções MIPS:

- `sll $t2, $s0, 4 # Desloca 4 bits à esquerda: $t2 <<4 $s0`

- `srl $t4, $t2, 4 # Desloca 4 bits à direita : $t4 >>4 $t2`

REPRESENTANDO INSTRUÇÕES: DESLOCAMENTO

✓ Representação de **instruções de computador** (processador):

- Considere os **exemplos MIPS de deslocamento** (shift):

```
sll $t2, $s0, 4 # Desloca 4 bits à esquerda: $t2 <<4 $s0
```

```
srl $t4, $t2, 4 # Desloca 4 bits à direita : $t4 >>4 $t2
```

- Temos as seguintes representações em linguagem de máquina

Nome	Formato	Campos						Hexa
sll	R	0	0	16	10	4	0	
Bin/Hexa		000000	00000	10000	01010	00100	000000	00105100 ₁₆
srl	R	0	0	10	12	4	2	
Bin/Hexa		000000	00000	01010	01100	00100	000010	000A6102 ₁₆
Tamanho	-	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	Aritmético
Formato	R	op	-	rt	rd	shamt	Function	32 bits

REPRESENTANDO INSTRUÇÕES: LÓGICAS

- ✓ Representação de **instruções de computador** (processador):
 - Operações lógicas: booleanas
 - **MIPS implementa** 3 operações lógicas: **And, Or, Nor**
 - Operação AND
 - Operação lógica **AND bit a bit** entre dois operandos
 - **Exemplo, sendo:** `$t1 = 0100 1101 1100 0000`, `$t2 = 0110 1100 0000 0000`
`and $t0,$t1,$t2 # And lógico: $t0 = $t1 & $t2`
 - **Resulta em:** `$t0 = 0100 1100 0000 0000`

REPRESENTANDO INSTRUÇÕES: LÓGICAS

- ✓ Representação de **instruções de computador** (processador):
 - Operações lógicas: booleanas
 - Operação OR
 - Operação lógica **OR bit a bit** entre dois operandos
 - **Exemplo, sendo:** $\$t1 = 0100\ 1101\ 1100\ 0000$, $\$t2 = 0110\ 1100\ 0000\ 0000$
or $\$t0, \$t1, \$t2$ # Or lógico: $\$t0 = \$t1 \mid \$t2$
 - **Resulta em:** $\$t0 = 0110\ 1101\ 1100\ 0000$

REPRESENTANDO INSTRUÇÕES: LÓGICAS

✓ Representação de **instruções de computador** (processador):

- Operação NOT

- Operação lógica de **negação bit a bit** sobre um operando
- MIPS implementa a operação NOR (Not Or) em ter 2 operandos
- Realizar a operação **Nor onde um operando é 0 (zero) resulta em Not**
- Assim, MIPS usa **NOR** para **calcular: NOR e NOT**

- Exemplo, sendo:

`$t1 = 0100 1101 1100 0000, $t2 = 0110 1100 0000 0000, $zero = 0000 0000 0000 0000`

(1) `nor $t0,$t1,$t2` # **Nor** lógico: `$t0 = $t1 ~| $t2`

(2) `nor $t0,$t1,$zero` # **Not** lógico: `$t0 = $t1 ~| $zero`

- Resulta em:

(1) `$t0 = 1001 0010 0011 1111` ← `0100 1101 1100 0000 ~| 0110 1100 0000 0000`

(2) `$t0 = 1011 0010 0011 1111` ← `0100 1101 1100 0000 ~| 0000 0000 0000 0000`

REPRESENTANDO INSTRUÇÕES: LÓGICAS

✓ Representação de **instruções de computador** (processador):

- Considere os exemplos MIPS de deslocamento:

`and $t0,$t1,$t2` # And lógico: $\$t0 = \$t1 \& \$t2$

`or $t0,$t1,$t2` # Or lógico: $\$t0 = \$t1 \mid \$t2$

`nor $t0,$t1,$zero` # Not lógico: $\$t0 = \$t1 \sim \mid \$zero$

- Temos as seguintes versões em linguagem de máquina

Nome	Formato	Campos						Hexa
and	R	0	9	10	8	0	36	
Bin/Hexa		000000	01001	01010	01000	00000	100100	012A4024 ₁₆
or	R	0	9	10	8	0	37	
Bin/Hexa		000000	01001	01010	01000	00000	100101	012A4025 ₁₆
nor	R	0	9	0	8	0	39	
Bin/Hexa		000000	01001	00000	01000	00000	100111	01204027 ₁₆
Tamanho	-	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	Aritmético
Formato	R	op	rs	rt	rd	shamt	Function	32 bits

REPRESENTANDO INSTRUÇÕES: DESVIO

- ✓ Representação de **instruções de computador** (processador):
 - Operações de desvio (branch) condicional: **tomada de decisões**
 - Possibilita a capacidade de **selecionar trechos de código**, exemplo: `if`
 - Em MIPS existem duas **instruções de tomada de decisão**
 1. Baseado na **comparação de igualdade** (equal)
`beq reg1, reg2, L1`
 - **Compara** dois registradores, **desvia para L1** (deslocamento/**rótulo**) caso forem iguais
 2. Baseado na **comparação de desigualdade** (not equal)
`bne reg1, reg2, L1`
 - **Compara** dois registradores, **desvia para L1** (deslocamento/**rótulo**) caso forem diferentes
 - E uma instrução de desvio incondicional
 3. Não usa **comparação**, apenas desvia
`j L1`
 - **Desvia** incondicionalmente **para L1** (deslocamento/**rótulo**), semelhante ao comando `go to`

REPRESENTANDO INSTRUÇÕES: DESVIO

- ✓ Representação de **instruções de computador** (processador):
- Formato das instruções de desvio

Nome	Formato	Campos				Endereço
beq	I	OpCode = 4	Rs	Rt	Imediato ¹ WordAligned	PC+4+Imediato ¹
bne	I	OpCode = 5	Rs	Rt	Imediato ¹ WordAligned	PC+4+Imediato ¹
Tamanho		6	5	5	16	
j	J	OpCode = 2	Imediato ² PC-relativo WordAligned			PC (4bMSD) + Imediato ²
Tamanho		6	26 ²			

¹ Deslocamento *Instruction-relative* (Word Aligned), a partir da próxima instrução (deslocamento em nº de instruções)

² Endereço alvo *PC-relative* : 28b/4 → Word Aligned; Gerando 26b ← 28b>>2, conforme a tabela seguir:

PC 4-MSD	Alvo (IMM)	LSD: 0 (zero)
4 bits	26 bits	2 bits ³

² Obtém-se o endereço destino de 32b: concatenando PC(4bMSD) com IMM(26b>>2)

- Exemplo: PC: 60CO BED0 | Destino: 60CO BEB0 → Alvo-28bLSD: 0000 11000000 10111110 10110000
- PC(4bMSD) → 0110 | Alvo-26b: 26bLSD(6OFF BEB0) → 0000 11000000 10111110 10110000 → 30 2FAC
- Endereço final de 32 bits PC+(Alvo<<2): 01100000 11000000 10111110 10110000 → 60CO BEB0 ← *4

REPRESENTANDO INSTRUÇÕES: DESVIO

✓ Expressões e **instruções de computador** (processador):

- Exemplo:

- Programa/expressão:

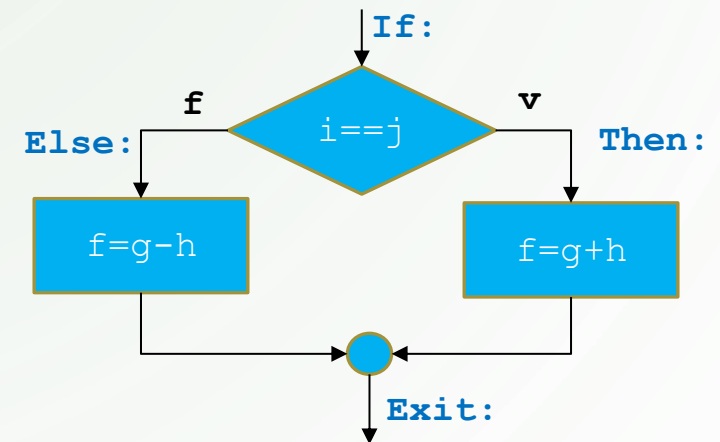
Endereços: f, g, h, i, j já carregados nos registradores $\$s0$ a $\$s4$, endereço inicial do código: FF00 4000

```
if (i == j) f = g + h; else f = g - h;
```

- Instruções MIPS:

```
If:   bne $s3,$s4,Else   # go to Else if i ≠ j
Then: add $s0,$s1,$s2    # f = g + h (pula if i ≠ j)
      j Exit            # go to Exit
Else: sub $s0,$s1,$s2    # f = g - h (pula if i = j)
Exit: ...
```

* Obs: rótulos em azul



REPRESENTANDO INSTRUÇÕES: DESVIO

- ✓ Representação de **instruções de computador** (processador):
- Considere o comando condicional (**if**) e respectivas instruções MIPS:

```
if (i == j) f = g + h; else f = g - h;
```

...0400	If:	bne \$s3,\$s4,Else	# go to Else if i ≠ j	*Código: endereço inicial F000 0400 ₁₆
...0404	Then:	add \$s0,\$s1,\$s2	# f = g + h (pula if i ≠ j)	
...0408		j Exit	# go to Exit	
...040C	Else:	sub \$s0,\$s1,\$s2	# f = g - h (pula if i = j)	
...0410	Exit:	...		

- Temos as seguintes versões em linguagem de máquina (f:s0,g:s1,h:s2,i:s3,j:s4)

Nome	Formato	Campos						Hexa
bne	I	5	19	20	2 ¹			16740002 ₁₆
add	R	0	17	18	16	0	32	02328020 ₁₆
j	J	2	260 ² (26b)					08000104 ₁₆
sub	R	0	17	18	16	0	34	02328022 ₁₆

¹ Rótulo **If**: deslocamento *Instruction-relative*: 2 (3-1) instruções adiante (2*4=8B)

² Exemplo: PC: F000 0408 Alvo: F000 0410 (2 words adiante)

PC:4bMSD: 1111 Alvo-26b word: 00 00000000 00000001 00000100 (0104₁₆ → 260₁₀)

Endereço final de 32 bits: 11110000 00000000 00000100 00010000

J-format: 6+26b → 000010 0000 00000000 00000100 000100 → 00001000 00000000 00000001 00000100

REPRESENTANDO INSTRUÇÕES: DESVIO

✓ Expressões e **instruções de computador** (processador):

• Exemplo:

• Programa/expressão:

(valores i , k em $\$s3$, $\$s5$ e endereço mtx em $\$s6$ (variáveis e registradores))

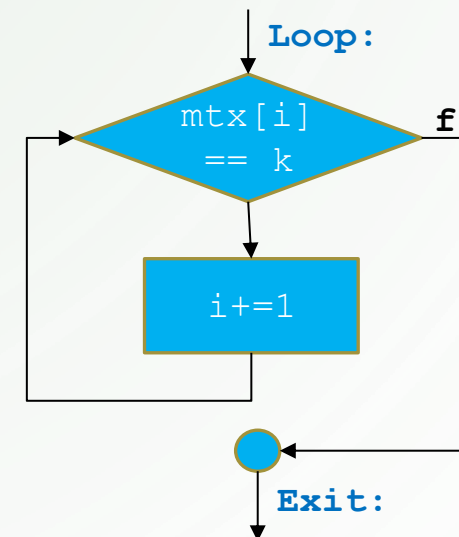
`while (  $mtx[i] == k$  )` *Código: endereço inicial 4000_{10}

`i+=1;`

• Instruções MIPS:

```
Loop: sll $t1,$s3,2      # Multiplica:  $t1 = i * 4$ 
      add $t1,$s6,$t1    # Ponteiro:  $\$t1 = mtx[i]$ 
      lw  $t0,0($t1)     # Leitura:  $\$t0 = mtx[i]$ 
      bne $t0,$s5, Exit # Desvia se:  $mtx[i] \neq k$ 
      addi $s3,$s3,1     #  $i = i + 1$ 
      j   Loop           # go to Loop
```

Exit: ...



REPRESENTANDO INSTRUÇÕES: DESVIO

- ✓ Representação de **instruções de computador** (processador):
 - Considere o laço (`while`) e respectivas instruções MIPS:
 - Temos as seguintes versões em linguagem de máquina

Nome	Formato	Campos						Hexa
sll	R	0	0	19	9	2	0	00134880 ₁₆
add	R	0	9	22	9	0	32	01364820 ₁₆
lw	I	35	9	8		0		8D280000 ₁₆
bne	I	5	19	21		2 ¹		15150002 ₁₆
addi	I	8	19	19		1		22730001 ₁₆
j	J	2			256 ²			08000100 ₁₆

Endereço	Código	Comentário
...0400	while (mtx[i] == k)	i+=1; (i, k em \$s3, \$s5 e mtx em \$s6)
	Loop: sll \$t1,\$s3,2	# Multiplica: t1 = i * 4
		# Código: endereço inicial F000 0400 ₁₀
...0404	add \$t1,\$t1,\$s6	# Ponteiro: \$t1 = mtx[i]
...0408	lw \$t0,0(\$t1)	# Carga: \$t0 = mtx[i]
...040C	bne \$t0,\$s5,Exit	# Salta se: mtx[i] ≠ k
...0410	addi \$s3,\$s3,1	# i = i + 1
...0414	j Loop	# go to Loop
...0418	Exit: ...	

¹ Deslocamento *Instruction-relative*: 2 (3-1) instruções adiante (2*4=8B)

² Exemplo: PC: F000 0414 Alvo: F000 0400 (5 words up)

PC:4bMSD: 1111 Alvo-26b word: 00 00000000 00000001 00000000 (0100₁₆ → 256₁₀)

Endereço final de 32 bits: 11110000 00000000 00000100 00000000

J-format: 6+26b → 000010 00 00000000 00000001 00000000
→ 00001000 00000000 00000001 00000000

REPRESENTANDO INSTRUÇÕES: DESVIO

- Operações de **desvio** em **geral** comparam **igualdade** ou **desigualdade**
- Entretanto, **por vezes** precisamos comparar outras magnitudes: **menor**, **maior**, etc
- Em MIPS usamos instruções **set-and-compare** para comparar estas magnitudes
 - A instrução **compara** (compare) **dois registradores** e **fixa** (set) um **3º** registrador para 1 (**true**) ou 0 (**false**)

1. Comparando **menor que** (less then) entre **dois registradores**

```
slt $t0, $s3, $s4    # $t0 = 1 if $s3 < $s4 → valores com sinal: signed
```

```
sltu $t0, $s3, $s4    # $t0 = 1 if $s3 < $s4 → valores sem sinal: unsigned
```

2. Comparando **menor que** com **imediato** (less then) entre um **registrador** e uma **constante**

```
slti $t0, $s2, 10     # $t0 = 1 if $s0 < 10 → valores com sinal: signed
```

```
sltiu $t0, $s2, 10     # $t0 = 1 if $s0 < 10 → valores sem sinal: unsigned
```

REPRESENTANDO INSTRUÇÕES: DESVIO

- Comparando magnitudes: **menor** e **maior**, com ou sem igualdade
- Usando **4 instruções** e **registrador Zero** conseguimos realizar **todos os testes** de comparação:
 - Instruções: `beq`, `bne`, `slt`, `slti` e o Registrador `$0`
 - Testes: `=`, `≠`, `<`, `>=`, `>`, `<=`
- Princípio de projeto: simplicidade
 - Duas instruções simples executam mais rápido que uma complexa
 - Exemplo: testar `<` e `=` individualmente ao invés de `<=` conjuntamente
 - Usamos conjuntamente:
 - `slt|slti` para `>` ou `<`
 - `beq` ou `bne` para `=`
 - `$0` para ajuste da lógica

REPRESENTANDO INSTRUÇÕES: DESVIO

- Comparando magnitudes: **menor** e **maior**, com ou sem igualdade
 - Considere `$s1` e `$s2` como os valores a serem comparados (nesta ordem), ex.: `$s1 <= $s2`
 - Todos os **desvios** (branch) são **tomados** caso a avaliação da **expressão** resulte **verdadeira** (True)
- Testes **diretos**: usado para desviar o fluxo de execução para rotinas (código) que tratam a condição (V) do teste
 - Se **verdadeiro**: desvia para o trecho de código que trata a condição do teste (original)
 - Se **falso**: segue o fluxo normal de processamento (próxima instrução)

1. Igual: `beq $s1,$s2,L1` # if `s1=s2` → L1

2. Diferente: `bne $s1,$s2,L1` # if `s1≠s2` → L1

3. Menor: `slt $t0,$s1,$s2 bne $t0,$0,L1` # `t0=1` if `s1<s2` e if `t0 !F` → L1

4. Maior igual: `slt $t0,$s1,$s2 beq $t0,$0,L1` # `t0=1` if `s1<s2` e if `t0 =F` → L1

5. Maior: `slt $t0,$s2,$s1 bne $t0,$0,L1` # `t0=1` if `s2<s1` e if `t0 !F` → L1

6. Menor igual: `slt $t0,$s2,$s1 beq $t0,$0,L1` # `t0=1` if `s2<s1` e if `t0 =F` → L1

- Os casos (1, 2) são complementares entre si, da mesma forma que (3, 4) e (5, 6)
- Exercício: teste os 6 casos acima com cada uma das 3 condições seguintes:
 - a) `$s1=3 $s2=3` b) `$s1=2 $s2=4` c) `$s1=4 $s2=2`

REPRESENTANDO INSTRUÇÕES: DESVIO

- Comparando magnitudes: **menor** e **maior**, com ou sem igualdade
 - Considere `$s1` e `$s2` como os valores a serem comparados (nesta ordem), ex.: `$s1 <= $s2`
 - Todos os **desvios** (branch) são **tomados** caso a avaliação da **expressão** resulte **verdadeira** (True)
- Testes **complementares**: usado em estruturas como: seleção condicional (if) e laços de repetição (while, do-while, for)
 - Se **verdadeiro**: desvia para um trecho de código que **NÃO** trata a condição **original** do teste
 - Se **falso**: segue o fluxo de processamento (próxima instrução), tratando a condição **original** do teste

1. Igual: `bne $s1,$s2,L1` # if `s1≠s2` → L1
2. Diferente: `beq $s1,$s2,L1` # if `s1=s2` → L1
3. Menor: `slt $t0,$s1,$s2 beq $t0,$0,L1` # `t0=1` if `s1<s2` e if `t0 =F` → L1
4. Maior igual: `slt $t0,$s1,$s2 bne $t0,$0,L1` # `t0=1` if `s1<s2` e if `t0 !F` → L1
5. Maior: `slt $t0,$s2,$s1 beq $t0,$0,L1` # `t0=1` if `s2<s1` e if `t0 =F` → L1
6. Menor igual: `slt $t0,$s2,$s1 bne $t0,$0,L1` # `t0=1` if `s2<s1` e if `t0 !F` → L1

- Os casos (1, 2) são complementares entre si, da mesma forma que (3, 4) e (5, 6)
- Exercício: teste os 6 casos acima com cada uma das 3 condições seguintes:
 - a) `$s1=3 $s2=3`
 - b) `$s1=2 $s2=4`
 - c) `$s1=4 $s2=2`

REPRESENTANDO INSTRUÇÕES: DESVIO

- Comparando magnitudes: **menor** e **maior**, com ou sem igualdade
- Comparativo entre testes diretos e complementares:

Teste	Direto	Complementar
=	beq \$s1,\$s2,L1 # if s1=s2 → L1	bne \$s1,\$s2,L1 # if s1≠s2 → L1
≠	bne \$s1,\$s2,L1 # if s1≠s2 → L1	beq \$s1,\$s2,L1 # if s1=s2 → L1
<	slt \$t0,\$s1,\$s2 bne \$t0,\$0,L1 # t0=1 if s1<s2 e if t0 !F → L1	slt \$t0,\$s1,\$s2 beq \$t0,\$0,L1 # t0=1 if s1<s2 e if t0 =F → L1
>=	slt \$t0,\$s1,\$s2 beq \$t0,\$0,L1 # t0=1 if s1<s2 e if t0 =F → L1	slt \$t0,\$s1,\$s2 bne \$t0,\$0,L1 # t0=1 if s1<s2 e if t0 !F → L1
>	slt \$t0,\$s2,\$s1 bne \$t0,\$0,L1 # t0=1 if s2<s1 e if t0 !F → L1	slt \$t0,\$s2,\$s1 beq \$t0,\$0,L1 # t0=1 if s2<s1 e if t0 =F → L1
<=	slt \$t0,\$s2,\$s1 beq \$t0,\$0,L1 # t0=1 if s2<s1 e if t0 =F → L1	slt \$t0,\$s2,\$s1 bne \$t0,\$0,L1 # t0=1 if s2<s1 e if t0 !F → L1

REPRESENTANDO INSTRUÇÕES: DESVIO

- Exemplos do comandos de seleção (if) usando comparações: <, >=, >, <= * e o assembly equivalente
* i, j, g, h, f estão em \$s1 a \$s5 respectivamente

```
if (i < j) f = g + h; else f = g - h;  
If:    slt $t0,$s1,$s2    # t0=(s1<s2)?1:0  
      beq $t0,$0,Else    # if t0 =F goto Else  
      add $s5,$s3,$s4    # f=g+h  
      j Exit            #  
Else:  sub $s5,$s3,$s4    # f=g-h  
Exit:  ...
```

```
if (i >= j) f = g + h; else f = g - h;  
If:    slt $t0,$s1,$s2    # t0=(s1<s2)?1:0  
      bne $t0,$0,Else    # if t0 !F goto Else  
      add $s5,$s3,$s4    # f=g+h  
      j Exit            #  
Else:  sub $s5,$s3,$s4    # f=g-h  
Exit:  ...
```

```
if (i > j) f = g + h; else f = g - h;  
If:    slt $t0,$s2,$s1    # t0=(s2<s1)?1:0  
      beq $t0,$0,Else    # if t0 =F goto Else  
      add $s5,$s3,$s4    # f=g+h  
      j Exit            #  
Else:  sub $s5,$s3,$s4    # f=g-h  
Exit:  ...
```

```
if (i <= j) f = g + h; else f = g - h;  
If:    slt $t0,$s2,$s1    # t0=(s2<s1)?1:0  
      bne $t0,$0,Else    # if t0 !F goto Else  
      add $s5,$s3,$s4    # f=g+h  
      j Exit            #  
Else:  sub $s5,$s3,$s4    # f=g-h  
Exit:  ...
```

INTERFACE: ENTRADA E SAÍDA

- Computadores **iniciaram** trabalhando com **números**
- Posteriormente **evoluíram** para tratar **textos**
- **Computadores são numéricos** por essência, assim **precisam** de uma interface para **suportar “não numéricos” (textos, símbolos)**
- Esta interface foi implementada como um **conjunto de símbolos** representados por **8 bits**
- Como 1 Byte possui 8 bits, temos 2^8 (256) símbolos diferentes
- Surge a tabela American Standard Code for Information Interchange (**ASCII**)

INTERFACE: ENTRADA E SAÍDA

- Tabela ASCII: subconjunto contendo os símbolos alfanuméricos

ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter
32	space	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	\$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

- Símbolos imprimíveis iniciam em 32 e terminam em 126
- Letras minúscula e maiúscula diferem entre si por 32 unidades (20_{16})
ex: **A**=65 (41_{16}), **a**=97 (61_{16}), o que simplifica comparações para ordenação

INTERFACE: ENTRADA E SAÍDA

✓ Instruções para manipulação de caracteres:

- Existem 2 instruções MIPS para manipular caracteres (8b):
leitura e escrita (I/O)

- Leitura de 1 byte da memória principal

```
lb $t0,0($s1)           # Lê um byte do endereço de memória indicado por s1 colocando em (LSD)t0
```

- Escrita de 1 byte da memória principal

```
sb $t0,0($s1)           # Escreve o byte (LSD)t0 no endereço de memória indicado por s1
```

- EM geral, caracteres estão inseridos em cadeias (strings)
- Representação em linguagem de máquina das instruções I/O MIPS para bytes (caracteres)

Nome	Formato	Campos				Hexa
lb	I	32	17	8	0	82280000 ₁₆
sb	I	40	17	8	0	A2280000 ₁₆
Formato	I	Op	Rd	Rt	Imediato	

INTERFACE: ENTRADA E SAÍDA

✓ **Instruções** para manipulação de **caracteres**:

- Formas de **representação** de **cadeias** (strings):

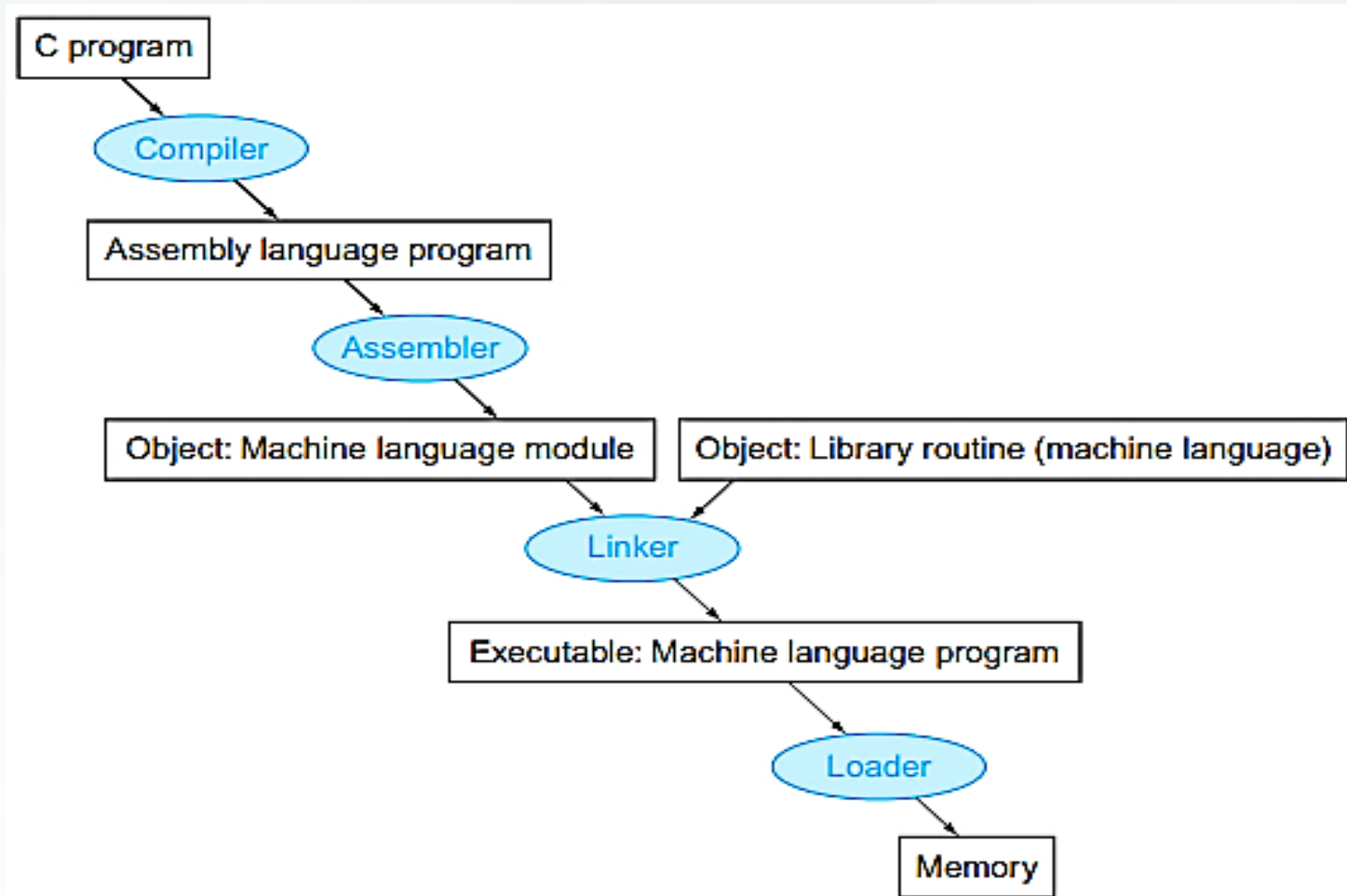
- a. A 1ª posição da cadeia indica o seu tamanho
- b. A cadeia possui uma variável extra que indica o seu tamanho
- c. A última posição da cadeia possui um caractere de terminação,
exemplo: a linguagem C usa o valor absoluto zero (0x0) como terminador de cadeia

- A linguagem C armazena a cadeia “Texto” usando 6 (5+1) bytes, exemplo:

$(84\ 101\ 120\ 116\ 111\ 0)_{10}$ **ou** $(54\ 65\ 78\ 74\ 6H\ 00)_{16}$

CÓDIGO: DO FONTE AO BINÁRIO

- ✓ Do programa **fonte** (source) às **instruções** de máquina (binário):
- Precisamos de várias etapas para chegar ao binário a partir do código fonte:



CÓDIGO: DO FONTE AO BINÁRIO

✓ Do programa **fonte** (source) ao código executável (binário):

- Do fonte ao binário: etapas

- **Compilador:**

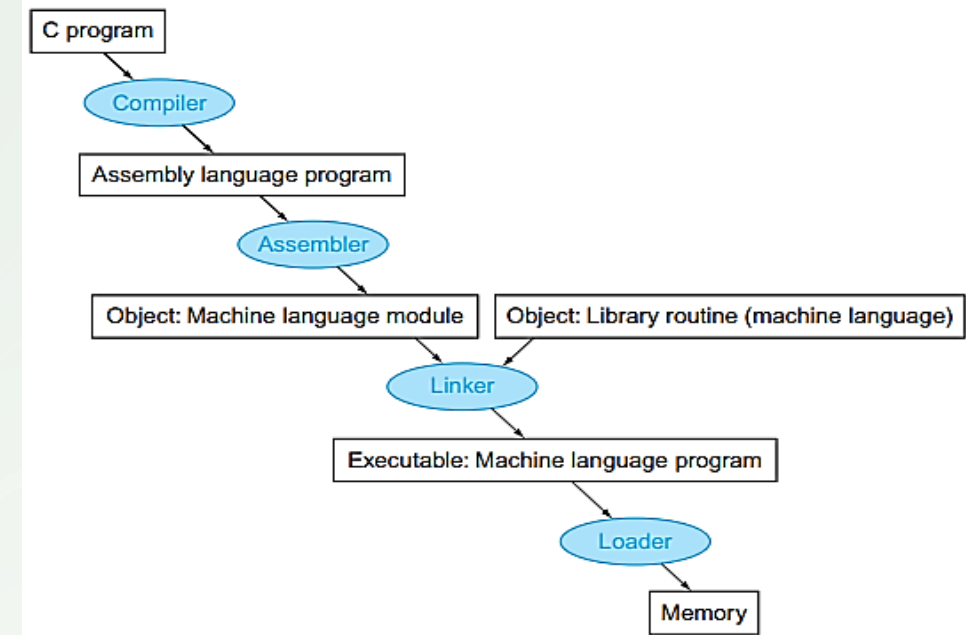
- Recebe o programa fonte (texto) e o transforma em instruções Assembly

- **Assembler:**

- Recebe instruções Assembly e as converte em binário, porém não resolve completamente todos os endereços

- **Linker:** combina programas binários (principal e códigos de bibliotecas) em um único programa executável, os endereços são relativos ao programa

- **Loader:** lê o programa executável e o carrega na memória principal, trata os endereços de forma absoluta



CÓDIGO SWAP COMO UM EXEMPLO

- Programa SWAP: troca valores entre duas posições de memória

```
void swap(int v[], int k){  
    int temp;  
    temp = v[k];  
    v[k] = v[k+1];  
    v[k+1] = temp;  
}
```

- O programa acima:
 - Recebe dois parâmetros: uma matriz e um valor indicando a posição de troca
 - Salva o valor original da matriz em uma variável auxiliar
 - Transfere o valor da posição seguinte de memória para a anterior
 - Transfere o valor salvo para a posição seguinte do vetor

CÓDIGO SWAP COMO UM EXEMPLO

- Programa SWAP equivalente em instruções MIPS:
 - Endereço base de V está em \$a0 e o valor de k está em \$a1
 - temp está associado ao registrador \$t0
 - \$t1 e \$t2 são registradores auxiliares

```
void swap(int v[], int k){  
    int temp;  
    temp = v[k];  
    v[k] = v[k+1];  
    v[k+1] = temp;  
}
```

```
swap:                # ponto de entrada (endereço inicial)  
sll $t1,$a1,2        # $t1 = k*4, ajusta word alignment (32b)  
add $t1,$a0,$t1      # $t1 = v+(k*4), t1 aponta para v[k]  
lw  $t0,0($t1)       # $t0 = v[k], lê o valor em v[k] para t0 (temp)  
lw  $t2,4($t1)       # $t2 = v[k+1], lê o valor seguinte em v[k+1] para t2  
sw  $t2,0($t1)       # v[k] = $t2, escreve o valor seguinte de v sobre o anterior  
sw  $t0,4($t1)       # v[k+1] = $t0, escreve o valor anterior de v (temp) sobre o seguinte
```

REPRESENTANDO INSTRUÇÕES

✓ Revisão de **instruções de computador** (processador):

- Resumo e exemplos de instruções MIPS:

Formato	Campos					
Formato R Arit/Log/Set	op	rs	rt	rd	ShAmt (0)	Function
Tamanho bits	6	5	5	5	5	6
Formato I I-O/Imm	op	rs	rt	Immediate		
Tamanho bits	6	5	5	16 bits		
Formato R Shift	op	rs (0)	rt	Rd	shamt	Function
Tamanho bits	6	5	5	5	5	6
Formato J	op	PC-relative Address				
Tamanho bits	6	26				

Registrador	Nome
\$8 ... \$15	\$t0 ... \$t7
\$16 ... \$23	\$s0 ... \$s7
\$24 e \$25	\$t8 e \$t9
\$4 e \$5	\$a0 e \$a1

Instrução	Código operer.
add	0/32
sub	0/34
addi	8
and	0/36
or	0/37
nor	0/39
lw - lb	35 - 32
sw - sb	43 - 40
sll	0/0
srl	0/2
beq	4
bne	5
j	2
slt - sltu	0/42 - 0/43
slti - sltiu	10 - 11